

- MicroPython's Pico port offers modules for interacting with low-level chip-specific hardware.
- Detailed documentation can be found at <https://www.raspberrypi.com/documentation/microcontrollers/micropython.html>
- **Arduino**
 - Arduino is an open source platform comprising hardware and software, allowing quick production of interactive electronics projects.
 - Arduino employs its own programming language, akin to C++; however, for applications that require integration with sensors and other physical devices, Arduino can be used with Python.
 - The Firmata protocol is the standard method to control Arduino from PC. It enables serial communication with microcontrollers from software on a host computer, allowing it to receive digital and analogue inputs and transmit data to digital and analogue outputs.
 - To get started with Arduino and Python, visit <https://realpython.com/arduino-python/>
- **IoT system and sensors interaction**
 - **PySensors:** Use 'pip install PySensors' to install it.
 - lm-sensors (Linux monitoring sensors) is a free and open source application that provides temperature, voltage, and fan monitoring tools and drivers.
 - The PySensors Python module is used for ctypes bindings for *libsensors.so* from the lm-sensors project.
 - Reference: <https://pypi.org/project/PySensors/>
 - **esptool:** Install using 'pip install esptool'.
 - esptool is a Python-based, open source, platform-independent software for communicating with the Espressif SoC's ROM bootloader.
 - The toolbox for working with Espressif chips includes esptool.py, espfuse.py, and espsecure.py.
 - They can, for example, accomplish these tasks:
 - Binary data saved on flash can be read, written, erased, and verified.
 - Can read chip characteristics and other data relating to the chip, such as the MAC address or flash chip ID.
 - The one-time-programmable efuses can be read and written.
 - Make binary executable images that are ready for flashing.
 - Binary images can be analysed, assembled, and merged.
 - Reference: <https://pypi.org/project/esptool/>
 - **pyusb:** Install with 'pip install pyusb'.
 - pyusb simplifies communication with USB devices in Python.
 - It should function without any additional code in any Python >= 3.6 environment with ctypes and a pre-built USB backend library.
 - Reference: <https://pypi.org/project/pyusb/>
 - **pyserial:** Use 'pip install pyserial' to install it.
 - pyserial is a Python library responsible for serial port access.
 - It provides the same class-based interface for Python running on Windows, OSX, Linux, BSD (potentially any POSIX-compliant system), and IronPython on all supported platforms.
 - This allows it to communicate serially with devices like voltmeters, oscilloscopes, strain gauges, flow meters, actuators, and lights.
 - Reference: <https://pypi.org/project/pyserial/>
 - **pybluez:** Install with 'pip install pybluez2'.
 - This module is responsible for serial port access.
 - It provides the same class-based interface for Python running on Windows, OSX, Linux, BSD (potentially any POSIX-compliant system), and IronPython on all supported platforms.
 - Reference: <https://pypi.org/project/pybluez2/>
 - **gpiozero:** Install via 'pip install gpiozero'.
 - gpiozero provides a straightforward interface to GPIO devices on the Raspberry Pi.
 - Component interfaces are provided to make getting started with physical computing as simple as possible.
 - One may rapidly connect components together with very little code.
 - Reference: <https://pypi.org/project/gpiozero/>
 - **mraa**
 - mraa is a Python-compatible structural GPIO library.
 - Eclipse Mraa (Libmraa) is a C/C++ library with Java, Python, and JavaScript bindings for interacting with I/O pins and buses on various IoT and edge systems, providing an organised and reasonable API, where port names/numbering match the board that you are on.
 - The use of Libmraa does not bind you to any particular hardware. Since board detection is done at runtime, you can write portable code that works on all supported platforms. The most appealing aspect of this is that there is only one library for each individual device.
 - The goal is to make it easy for developers and sensor manufacturers to map their sensors and actuators on top of compatible hardware, and to allow high level languages and structures to govern low level communication protocols.
 - Reference: <https://github.com/eclipse/mraa/tree/master/examples/python>
 - **IoT communication protocols**
 - **sockets:** Install using 'pip install sockets'.
 - TCP/IP and UDP are communication transport layer protocols.